number of generations or until we reach *convergence* (which is when the average population's fitness has stopped improving significantly).

6.2.2. Evolution in practice

Before we dive into the reinforcement learning application, we'll run a super simple genetic algorithm on an example problem for illustrative purposes. We will create a population of random strings and try to evolve them toward a target string of our choosing, such as "Hello World!"

Our initial population of random strings will look like "gMIgSkybXZyP" and "adlBOM XIrBH." We'll use a function that can tell us how similar these strings are to the target string to give us the fitness scores. We'll then sample pairs of parents from the population weighted by their relative fitness scores, such that individuals with higher fitness scores are more likely to be chosen to become parents. Next we'll mate these parents (also called *crossing* or *recombining*) to produce two offspring strings and add them to the next generation. We'll also mutate the offspring by randomly flipping a few characters in the string. We'll iterate this process and expect that the population will become enriched with strings very close to our target; probably at least one will hit our target exactly (at which point we'll stop the algorithm). This evolutionary process for strings is depicted in figure 6.5.

Figure 6.5. A string diagram outlining the major steps in a genetic algorithm for evolving a set of random strings toward a target string. We start with a population of random strings, compare each to the target string, and assign a fitness score to each string based on how similar it is to the target string. We then select high-fitness parents to "mate" (or recombine) to produce children, and then we mutate the children to introduce new genetic variance. We repeat the process of selecting parents and producing children until the next generation is full (when it's the same size as the starting population).